# TM-V71 Remote

## User & Technical Manual

Kenwood TM-V71 web remote




Kenwood TM-V71(A/E) web remote + WebRTC audio + HackRF panadapter, for a Raspberry Pi.

Version 3.5 · 2026-10-16
github.com/CQ-DJ0SH/tmv71-remote

# Contents


**1 Overview** 3

**2 Features** 4

**3 Architecture** 5

Audio path 5

**4 Requirements & Hardware** 7

Wiring the USB sound card and the power switch 7

**5 Installation** 9

**6 Running over HTTPS** 10

**7 The Web Interface** 11

What a narrow window does 11

Band panels (VFO A / VFO B) 11

PTT & memory quick keys 12

MUTE and DROP 12

Audio (WebRTC/Opus) 13

HackRF waterfall 15

Selcall (classic 5-tone) 15

Digimodes (CW / RTTY / POCSAG / APRS) 15

APRS 16

Contact columns beside the console 17

Callsign recognition (Vosk) 17

Band scan 18

ASR contacts 19

Voice ID — an over without a callsign 21

Logbook (Wavelog + QRZ.com) 22

Settings 22

**8 Mobile App (PWA)** 25

**9 Trusted Certificate (Root CA)** 26

**10 Configuration** 27

**11 REST & WebSocket API** 28

Control & status 28

Memory channels 28

Audio 28

Digimodes & selcall 28

SDR & scan 29

Logbook 29

System & power 29

**12 Troubleshooting** 30

**13 Security** 31

**14 Credits & License** 32

# 1 Overview

TM-V71 Remote is a modern, dependency-light web remote control for the Kenwood TM-V71(A/E) dual-band FM transceiver, built around a direct serial driver. It gives full radio control in the browser, two-way browser audio over WebRTC/Opus, complete memory-channel management, an optional HackRF panadapter, classic 5-tone selective calling, a CW/RTTY/POCSAG digimodes decoder/encoder, a raw RX recorder, and optional off-air callsign recognition (Vosk). It installs as a Progressive Web App (PWA) and is designed to run on a Raspberry Pi.

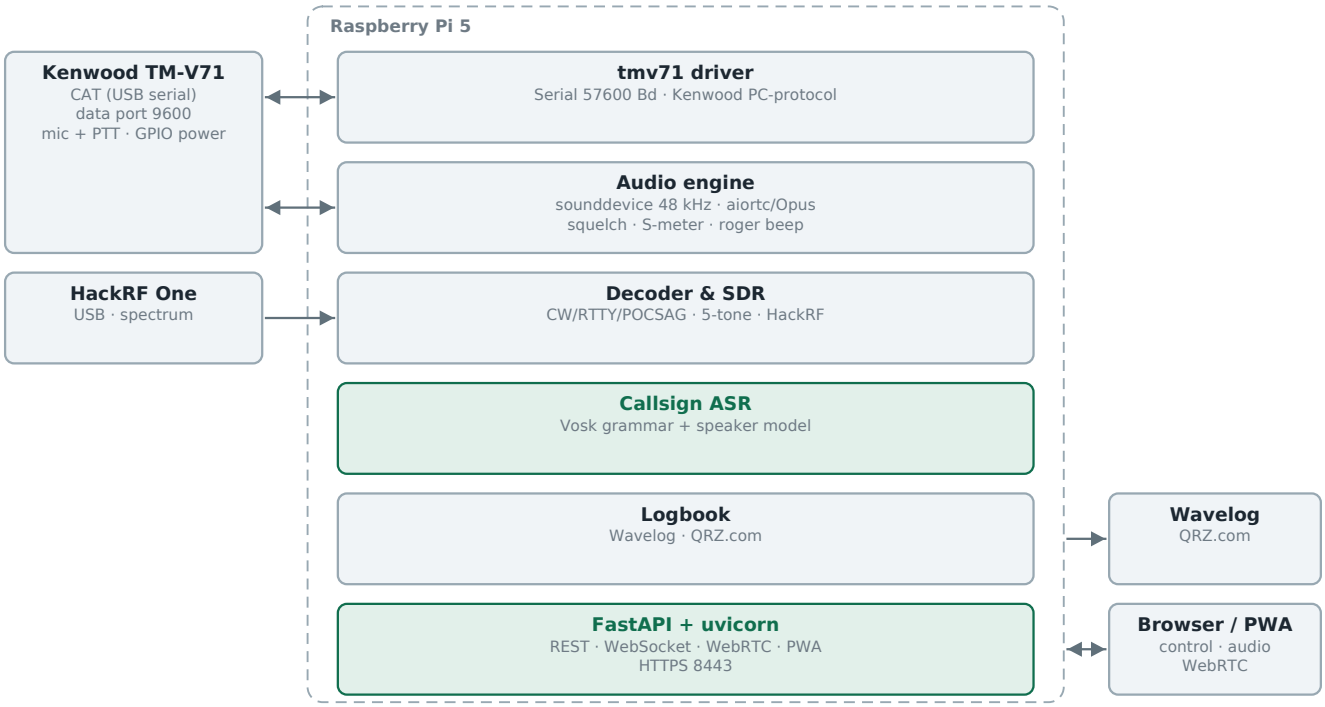
Unlike hamlib, whose TM-V71 backends are unreliable, this project speaks the radio's documented PC command set directly and exposes the radio's full feature set, including per-channel memory programming.

# 2 Features

- Full live control of both bands (A/B): frequency, VFO/memory mode, repeater shift & offset, CTCSS/DCS, step, control band, and PTT over CAT.
- Memory channels (CHIRP-level): read, write, delete, rename any of the 1000 channels, plus CSV import/export.
- Live status pushed to the browser over a WebSocket; transmit lights the UI.
- Two-way audio: direct WebRTC/Opus between browser and backend via aiortc; the mic feeds the radio only while PTT is engaged.
- Optional HackRF One waterfall: real-time panadapter (auto-following the tuned frequency) or a wideband sweep.
- Classic 5-tone selective calling (ZVEI-1/2, CCIR, EEA): call, decode, and mute RX until your own ID is received.
- CW (Morse), RTTY (Baudot/AFSK) and POCSAG paging (512/1200/2400 baud, numeric + alphanumeric, BCH FEC) decode + encode over the FM audio path; CW auto mode tracks the received speed and tone pitch, plus a button to decode a captured RX buffer off-line.
- Audio processing: RX de-emphasis (for a flat 9600/discriminator feed), BUSY-gated software squelch, TX AGC, and voice low-pass filters.
- Raw RX recorder with WAV download (e.g. to build ASR training data).
- Off-air callsign recognition (optional, Vosk): detects spoken German callsigns, verifies them against the BNetzA list (name/town/class, or VOID if unassigned), shown in the title bar and a toast.
- Installable PWA with a mobile landscape swipe-deck layout.
- Resilient operation: the phone screen is kept awake, browser audio auto-reconnects after a network glitch, and a backend watchdog releases a latched PTT if every client disappears.
- GPIO power switch, auto power-off, TX power, squelch, in-display S-meter.
- Two themes (dark/light); no build step for the UI.

# 3 Architecture

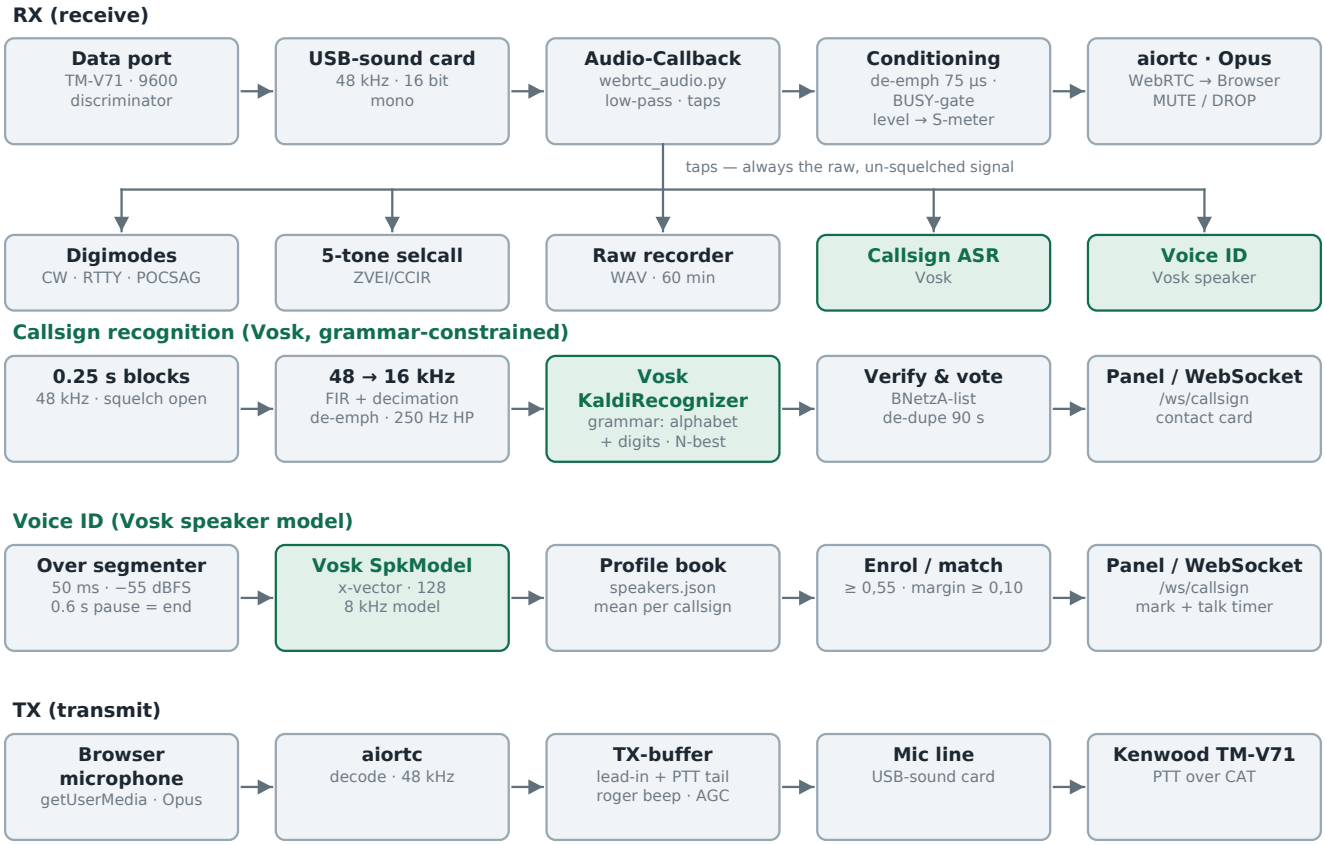
Everything below runs on the Pi. The radio is reached over two independent paths — CAT commands on the serial line, and audio on the data port through a USB sound card — and the browser sees only the one HTTPS address the backend serves.




Everything runs on the Pi; the browser gets control, status and audio over one HTTPS address.

## Audio path

One capture callback owns the received audio. What the browser hears is conditioned and squelched; every decoder is fed from a tap taken BEFORE that, on the raw signal, which is why the S-meter, the recorder and both Vosk chains keep working while MUTE or DROP is on. Vosk appears twice on purpose: the grammar-constrained decoder reads spoken callsigns, and a second, plain recognizer with the speaker model turns a whole over into a voiceprint (the grammar decoder runs with N-best alternatives, and in that mode Vosk returns no x-vector).




The decoders always sit on the raw signal — squelch, MUTE and DROP only affect what the browser hears.

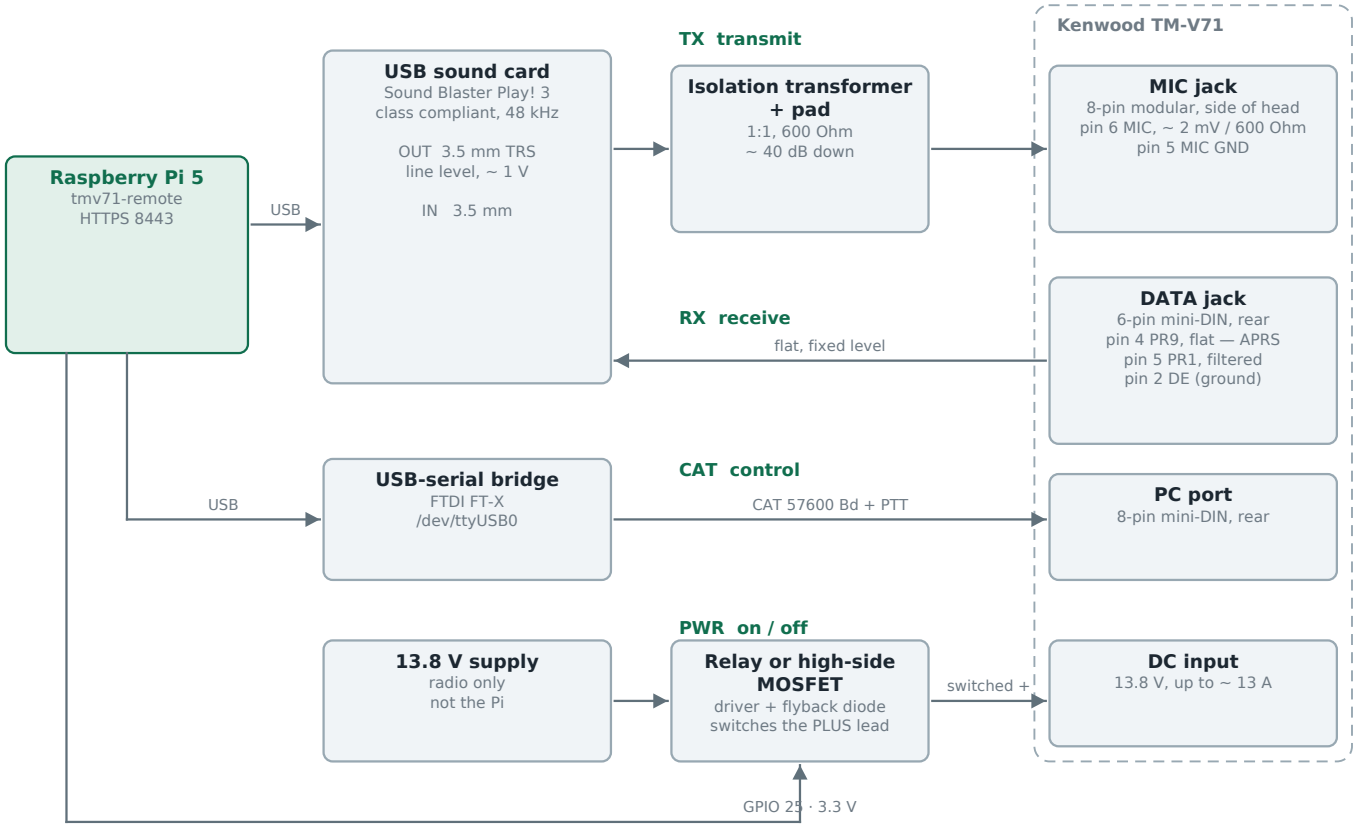
The backend owns the serial port directly (backend/app/tmv71.py). One FastAPI process serves the SPA/PWA, the REST control endpoints, the live-status WebSocket, and the WebRTC audio signalling — no extra services. Audio is 48 kHz / 16-bit / mono internally (Opus' native rate).

# 4 Requirements & Hardware

- Raspberry Pi (tested on Debian 13 / aarch64), Python 3.11+.
- Kenwood TM-V71(A/E) on a serial port (FTDI programming cable), 57600 baud.
- A USB sound interface wired to the radio (data port or mic/speaker), full-duplex.
- System packages: portaudio19-dev, swig + liblgpio-dev (optional GPIO).
- Optional: a HackRF One plus the hackrf host tools for the waterfall.
- Optional: vosk + the small German model (offline callsign recognition), and pypdf + the BNetzA Rufzeichenliste PDF (name/town/class + VOID check).

## Wiring the USB sound card and the power switch

The audio does not use one connector but two, and they sit on opposite sides of the radio: transmit audio goes into the mic jack on the side of the control head, received audio comes off the DATA jack on the rear panel. That split is not a preference, it follows from how the radio switches its paths — see the note under the diagram.




TX goes into the mic jack, not into the DATA jack: the radio only routes transmit audio from the DATA jack while a HARDWARE PTT keys it, and this station keys PTT over CAT. RX is taken from PR9, the flat 9600-baud pin: fixed level, untouched by the volume knob, and with neither de-emphasis nor the CTCSS filter in the way — the better feed for APRS. Set menu 519 (PC port baud) to 57600. The PLUS lead is the one that gets switched: Pi and radio already share ground through the audio and CAT cables, so a switch in the negative lead would simply be bridged by them.

- The transmit path needs to lose about 40 dB: the card puts out line level (~1 V), the mic input expects a few millivolts into 600 ohms. A 1:1 transformer in that lead also breaks the ground loop between the Pi's supply and the radio, which is what an alternator whine or a steady hum on your transmitted audio usually is.
- Both receive pins sit at a fixed level, independent of the volume knob, so what the Pi hears does not change when you turn the radio down. They differ in what the radio has

already done to the audio. PR9 (pin 4) is the flat output straight off the discriminator; PR1 (pin 5) has been through de-emphasis and the CTCSS filter.

- Take PR9 if APRS matters. De-emphasis tilts the two AFSK tones against each other — 2200 Hz arrives some 5 dB below 1200 Hz — and that twist costs decodes at the edge of the noise; the flat pin hands the demodulator both tones as they were sent. It is also the pin the rest of this program is built around: the software puts the de-emphasis back for voice (on by default), high-passes the sub-audible CTCSS tone the flat output passes, and gates the audio from the radio's BUSY line instead of relying on the squelch — which on the speaker path would chop a frame in half.
- If PR9 stays silent on your unit, set menu 518 (data speed) to 9600: on some radios that menu switches the receive path as well. It costs nothing here either way, because the transmit audio never touches the DATA jack.
- PTT is keyed over CAT on the PC port, so no PTT line is wired. Nothing in the audio cabling can key the radio.
- Power: the radio cannot be switched on over its serial port, so its 13.8 V line is switched instead, from a GPIO pin (BCM 25 by default, Settings > General). The PLUS lead is the one to break. Pi and radio already share ground through the audio and CAT cables, so a switch in the negative lead would be bridged by them — the radio would keep drawing its current through the audio screen, which is both wrong and dangerous.
- Size it for the transmitter, not the receiver: a TM-V71 draws around 13 A at 50 W. An automotive relay (20–30 A contacts) or a high-side P-channel MOSFET rated well above that, with short, thick leads.
- A GPIO pin cannot drive a relay coil — 3.3 V at a few milliamps against 80–150 mA. Between them belongs a logic-level N-MOSFET or a ready-made opto-isolated relay board, and a flyback diode across the coil; without it the coil's collapsing field puts a spike straight back into the driver. A board that switches when its input is pulled LOW is common, and supported: set the active level in Settings.
- Never power the Pi from the switched side. It would cut its own supply the moment it switched the radio off, and nothing would switch it back on.
- Reconfiguring the pin never toggles the radio: it is claimed without being driven. A restart is different, and it was measured here — the GPIO line is released when the process exits, the pin falls back to an input, and the radio loses its supply. The hardware cannot be asked afterwards what it was, so the service remembers the switch state and puts it back on startup. Auto power-off runs on the Pi itself and therefore keeps working with every browser closed.

# 5 Installation

```
sudo apt-get install -y portaudio19-dev python3-venv swig liblgpio-dev
git clone https://github.com/CQ-DJ0SH/tmv71-remote.git
cd tmv71-remote/backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

For a reboot-proof setup, install the systemd unit from the deploy/ directory. The service runs uvicorn with TLS on port 8443.

# 6 Running over HTTPS

Browser microphone access (getUserMedia) and the PWA service worker require a secure context, so the server runs over HTTPS. A quick self-signed certificate is enough on the desktop (accept the warning once); for installing the PWA on a phone you need a trusted certificate (see chapter 9).

```
cd backend && mkdir -p certs
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
  -keyout certs/key.pem -out certs/cert.pem -subj "/CN=tmv71-remote" \
  -addext "subjectAltName=IP:<pi-ip>,DNS:localhost,IP:127.0.0.1"
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8443 \
  --ssl-keyfile certs/key.pem --ssl-certfile certs/cert.pem
```

Open https://<pi-ip>:8443/ and accept the certificate once.

# 7 The Web Interface

## What a narrow window does

The panels have a size they were drawn in and they keep it. A band panel cannot shrink anyway — its digit tuner is some 500 px of fixed parts — while the PTT and audio panels beside it have no such backbone: their buttons wrap and their sliders give way. Left to themselves, the two below squeezed while the two above stood still, which is what made a narrow window look lopsided. All four now share one floor of 540 px, and below about 1160 px of window they stack into a single full-width column instead of being cramped side by side.

The title bar gives way in the same spirit. Its middle plate keeps its own width, so it used to run over the callsign on one side and the clock on the other; its lettering and padding now shrink with the window, as far as the nameplate screws allow. The clock itself disappears below 1400 px — it is the one thing up there nobody operates, and every device showing this page has a clock of its own a glance away. Below 760 px the mobile deck takes over entirely (chapter 8).

## Band panels (VFO A / VFO B)

Each band shows the frequency on a 7-segment display with two stacked meters under a shared S-scale: a real S-meter (S0–S9) in the active band's colour, and below it the AF level / mic-modulation bar (1 s peak-hold). Plus controls for VFO/memory mode, CTRL/PTT band selection, TX power, squelch (remembered per band across power cycles), repeater shift/offset, tone and bandwidth. The digit tuner lets you click bars above/below each digit to step the frequency; AIR Band tunes band A to the 118–137 MHz air band (receive-only).

The S-meter is derived from FM quieting: the high-band noise on the flat RX is inverse to signal strength (loud hiss = no signal, full quieting = strong signal), so it estimates the received signal even though the TM-V71 sends no numeric RSSI over CAT — only a binary BUSY status. It is a relative quieting estimate: a readable signal reads mid/upper scale, a strong local signal saturates at S9, and it snaps back to S0 the moment the carrier drops.

### Band lock (CTRL / PTT)

CTRL moves the control band — what the spinner, the frequency entry and a memory recall act on — and PTT moves the transmit band. Both are a single tap and both reach the radio at once, so a stray tap leaves you tuning the band you are not watching, or keying the one you did not mean to. On a phone that is easy to do: the two buttons sit in the band head, a thumb's width from the swipe you were aiming for. The padlock in the title bar freezes the pair where they stand. All four buttons then carry a small padlock, a tap on one says what is going on instead of doing something, and a press held for about 0.7 s still switches the band — the lock is there to stop the accident, not to make a band change something you have to unlock first. It is remembered per browser, so the phone can run locked while the PC in the shack does not.

## PTT & memory quick keys

Hold the large PTT button (or the space bar) to transmit; PTT-LOCK latches transmit. PTT and PTT-LOCK require connected audio (there is no mic otherwise) — they are disabled while audio is off and released automatically if audio disconnects. ROGER adds a descending three-tone beep (d6 · a5 · e5, i.e. 1175/880/659 Hz, 60 ms each with 30 ms gaps) on release, at the level set in the audio settings; while transmitting the button shows a count-up timer (MM:SS). The 1750 Hz button arms a tone-call. Memory quick keys recall channels 0–16 (M0–M9 in the left column, M10–M16 in the right; the loaded channel's key glows). With band A in the air band they switch to 50–66, a bank of their own, so air-band presets and the ordinary ones do not share slots; below them the right column sends three DTMF memories (0–2). A status line shows per-band BUSY, the ASR state and the live RX/TX gain (in the PWA; the desktop shows the transmit hint). On mobile, mini RX/TX VU bars with peak-hold flank the button.

## MUTE and DROP

Two buttons flank the “PTT → BAND x” caption and silence the received audio, differing only in when they stop:

- MUTE (left) is the plain one: it stays until it is pressed again. The label reads MUTED while it is on.
- DROP (right) blanks the transmission in progress and lifts itself as soon as that station stops keying — for an interferer or a rag-chew you do not want to hear, without having to remember to un-mute afterwards.

Both glow in the colour of the transmit band while armed. Three details are worth knowing:

- DROP releases on the falling edge of that band's BUSY signal, not merely because the channel is quiet: arming it on an idle channel keeps the audio muted until the end of the next transmission, rather than un-muting again on the next status update.
- DROP also lifts after three seconds without speech, even while BUSY is still up — the permanent carrier of a repeater would otherwise keep the audio muted indefinitely. What counts as silence is measured against the noise floor, not against a fixed level, because on a flat data feed a pause is not silence at all: it is the quieted noise of a carrier that is still standing. Measured here, speech sat at -20 dBFS and a three-second speaker change at -37.5, while the quietest frame of a whole 36-second recording never reached -39 — a fixed threshold of -55 dBFS could not fire once. The floor is therefore tracked (5th percentile of the last half minute) and anything within 8 dB of it is a pause; a receiver that does fall properly silent still releases on the absolute rule. The three seconds are what keeps a breath between words from counting as a pause.
- The same pause lets go of the contact card: the talk timer stops and the mark — including one the voice recognition put there — is released, by the same rule and on the same poll as DROP. Through a repeater BUSY stands for the whole round, so without it the clock would credit the next station's over to the previous one, and the voice mark would sit on a station that had finished. The timer stops as of the START of the pause: three seconds of silence are nobody's talk time. This used to be a rule of its own, the absolute one, which on a flat feed never fired — so it let go only when the carrier dropped.
- Muting is done in the browser, on the audio element. The S-meter, the raw RX recorder

and the callsign recognition keep receiving the signal — only what you hear is silenced.
- A three-tone chime confirms each change (e5 · a5 · d6 rising on release, the same triad reversed when muting), so the two are told apart without looking. It is generated in the browser and can therefore never be mixed into the transmitted audio, unlike the roger beep, which deliberately is.

## Audio (WebRTC/Opus)

Open the AUDIO panel, pick the RX band with the RX-A/RX-B switch, click CONNECT and allow the microphone. RX and mic levels are shown live, with the WebRTC RX/TX data rate in the graph corner. Controls: RX/TX gain (with a recommended-default tick), MIC (mic test — meters the mic without keying, records while on and replays your audio over RX when switched off; RX is muted during the test), AGC (automatic TX level), and a small recorder — ● REC / ▶ PLAY plus a WAV download of the raw, un-squelched RX feed (up to 60 min; e.g. to build ASR training data; the downloaded file is named with the date and time it was saved). Audio timing (TX buffer / TX trail / RX buffer) and the USB card mixer are in Settings > Audio. The link auto-reconnects after a network glitch and is restored on the next launch.

### Sharing the radio with a TNC (APRS, packet)

RX-A/RX-B normally moves the radio's external data band (menu 38), because the data connector carries the audio of one band only and that is where the Pi's sound card usually sits. A station that has given the data connector to a KISS TNC — APRS messaging, weather beacons, packet — needs that field left exactly where the TNC wants it: one write to it takes the TNC's audio away. Settings > Audio > “Disable audio channel switching” does that. Nothing in the app then writes menu 38; RX-A/RX-B still works but moves only the app's own listen band, which is what the software squelch, the S-meter feed and the recognition's band label go by, and the label beside it reads LISTEN rather than BAND to say so. The setting lives on the Pi, so it holds for every browser and across restarts.

The wiring that goes with it: the Pi's sound card moves to the speaker jack SP1, with an attenuator (a pot, or two resistors) because that output is made for a loudspeaker. Per the schematic a dummy plug belongs in SP2, which breaks the bridge between the two jacks. The speaker output carries both bands, so voice works whatever the data band says — but it is behind a low-pass filter, so the digimode and selcall decoders in this app cannot work off it. They need the flat data connector, which the TNC now has.

The TX buffer bounds how much microphone audio may queue up for the radio. It has to be larger than the clusters the browser delivers in: measured here with a 40 ms cap, 2.3 s of silence were inserted for want of samples while 2.5 s were discarded at the cap — starving and overflowing at the same time, heard as dropouts. 150 ms is the default and the slider stops at 80 ms, because below that no setting can work. Half of it (40–80 ms) is collected as a cushion before play-out starts.

TX trail is how long the radio stays keyed after the button is released, so the last syllables are not chopped: it covers the frames still in flight over WebRTC at that moment. Once that time is up the over is treated as finished — no further microphone audio is accepted, the silence at the end of it is dropped (below -40 dBFS, with 40 ms kept after the last sound so a soft final consonant survives), and only what was actually spoken plays out before the key goes up. This matters because the browser microphone streams continuously and is not

gated by the PTT button: without that cut-off the queue never ran empty and every over was held open for the full timeout, half a second of room noise on the air.

A digital transmission (CW, RTTY, POCSAG, APRS beacon) owns the mic line for as long as it is keyed. Queued microphone audio is discarded when it starts, nothing new is taken while it runs, and the key goes up the moment its last symbol has gone out — with no voice tail and no roger beep, which is a voice-mode habit that would only corrupt the end of the transmission.

The RX buffer (Settings > Audio, 60 ms by default) is a jitter buffer between the sound card and the WebRTC track, and it is worth knowing why it exists. The card produces a 20 ms block on its own clock; the track emits a frame every 20 ms on the backend's clock. Two independent clocks sampling each other means that whenever one runs a few milliseconds early or late, a block goes out twice or is skipped — and every such seam is an audible click. Under load (the callsign recogniser runs continuously, and each over costs another ~1.3 s for its voiceprint) that is frequent enough to sound like crackle. Three 20 ms blocks absorb it; the delay they add is not noticeable in a QSO. Raise it if the received audio still crackles, lower it for the shortest possible delay. Each listening browser gets its own queue, capped so a client that falls behind loses its oldest blocks instead of accumulating delay; a queue that runs dry refills to the target before playing out again, because limping along one block deep means the next hiccup clicks as well.

RX conditioning (Settings > Audio): a RX de-emphasis (adjustable time constant, on by default) restores natural voice tone when the audio comes from a flat discriminator / 9600-baud data output; a fixed ~180 Hz high-pass on the listen path removes the CTCSS/PL sub-audible tone (67–254 Hz) and DC hum that this flat output passes and de-emphasis would otherwise lift (audible as a low speaker hum), while leaving voice untouched; a BUSY-gated software squelch re-applies muting from the radio's own busy status for that always-open output; and TX/RX voice low-pass filters (≤ 3.5 kHz) tame hiss. The decoders always receive the un-squelched, un-filtered signal.

TX modulation (Settings > Audio). Two stages can be switched into the transmit path, followed by a peak limiter that is always active behind them.

- COMPRESSOR (off by default) raises the AVERAGE modulation: quiet syllables and trailing words come up by as much as 9 dB while loud bursts are held back. Measured on a 1 kHz tone, an input range of −45…−1 dBFS leaves as −36…−9 dBFS, so 44 dB of speech dynamics become 27 dB. It acts within a syllable — unlike the AGC, which rides the overall level over seconds; both may be on. Below −52 dBFS nothing is lifted, so the pauses between words keep their room noise in the background instead of being pumped up to speech level.
- PRE-EMPHASIS (on by default) lifts the treble by 6 dB/octave — the exact inverse of the RX de-emphasis, using the same time constant, and within 0.2 dB of the analogue curve (+4.6 dB at 3 kHz). Switch it on ONLY if the transmit audio goes into a FLAT input, i.e. the 9600-baud data port: the radio's mic input and its 1200-baud input pre-emphasise by themselves, and doing it twice sounds shrill. The voice low-pass runs automatically while it is on — without it the lift would send hiss above the voice band as well (+11 dB at 8 kHz, −44 dB with it).
- The LIMITER holds peaks at −1 dBFS whenever either stage is on. It is not decoration: the emphasis lifts treble before the band-limiting low-pass, so a hot microphone clipped

in 16-bit — 1.9 % of samples at a −1 dBFS peak — and hard clipping there lands as broadband crackle INSIDE the voice band, where no filter can take it out again. The limiter reduces gain over 2.5 ms slices and ramps between them; measured across the chain, a 0 dBFS input leaves at −1.0 dBFS with not a single clipped sample.

Try either with MIC TEST: the replay runs through the same chain, so it is heard without transmitting.

### Microphone path — why a phone can transmit quietly

The browser's own processing — echo cancellation, noise suppression and automatic gain — is what makes a gain of 1 correct on a desktop: it levels the microphone before the audio is ever encoded. On iOS that same processing is Apple's voice unit, and it comes as one piece: switching it off costs the levelling too, which is why a phone can transmit far more quietly than its own recorder app, even with TX GAIN at the top. Switching it on shapes the speech and pulls a Bluetooth headset onto the mono HFP profile, taking the received audio down with it.

Neither is right for every station, so it is a setting: Settings > Audio > Microphone path. BROWSER DEFAULT leaves the processing to the browser, OFF forces all three off. Android starts at OFF — that is where a headset dropping to HFP was first seen — while iOS and the desktop start at BROWSER DEFAULT. The choice is remembered per browser, and changing it re-takes the microphone, so the audio link reconnects for a moment.

With OFF on an iPhone or iPad the signal arrives quiet: use TX GAIN or the AGC to make it up, and judge the result with MIC TEST, which runs the replay through the same chain without transmitting. If a Bluetooth headset is in use, listen for the receive audio going dull after a switch to BROWSER DEFAULT — that is the headset falling back to HFP.

Bluetooth headsets: transmit audio is captured from the phone's built-in microphone (not the headset's), so the headset stays on the A2DP profile and receive audio keeps coming through in good quality. Using the headset mic would force Android onto the mono HFP/SCO profile and, on many phones, leave RX stuck until Bluetooth is toggled.

## HackRF waterfall

If a HackRF One is connected, this panel shows a live spectrum stacked over a waterfall: a panadapter centred on the tuned frequency (auto-following) or a wideband sweep. Receive-only; LNA/VGA gains and a display level are adjustable.

## Selcall (classic 5-tone)

Send and decode classic selective calls (ZVEI-1/2, CCIR, EEA). Enter a 5-digit CALL code and press CALL (keys PTT). Enter your own ID and press MUTE to silence RX until your ID is received — then it un-mutes automatically. Over FM this is AFSK; use a dummy load when setting up.

## Digimodes (CW / RTTY / POCSAG / APRS)

Switch between CW (Morse), RTTY (Baudot/AFSK) and POCSAG paging. DECODE shows received text; type into the field and SEND to transmit (keys PTT); the CW text input is forced upper case. Parameters: CW WPM/pitch — with an AUTO mode that tracks both the

received speed and tone pitch (shown live on the sliders) and rejects voice/noise so it locks onto the CW even after a spoken ident; RTTY baud/shift/mark; and POCSAG baud (512/1200/2400), RIC, function and numeric/alphanumeric (auto-detected on RX) — e.g. monitor DAPNET on 439.9875 MHz with per-page RIC/FUNC/timestamp output. The REC button decodes the raw RX recorder buffer off-line in the current mode. Over the FM radio this is MCW / AFSK / FSK — not native HF modes.

## APRS

The fourth digimode, in software: the TM-V71 has no built-in TNC — that is the TM-D710 — so the whole stack runs on the Pi. Feed it the flat 9600-baud data output rather than the speaker path — pin 4 (PR9) of the rear DATA jack, see the wiring diagram in chapter 4: no de-emphasis, and no squelch chopping a frame in half.

- RECEIVE decodes Bell 202 AFSK (1200 baud, mark 1200 Hz, space 2200 Hz) into AX.25 and then APRS: uncompressed and compressed positions, MIC-E — which most mobile stations send — status, messages, objects, telemetry and weather. Each frame becomes one line with source, path, position, course, speed and comment; DEBUG adds a second line with the frame type, symbol, length and the raw payload. The panel head counts frames, damaged frames, distinct stations and noise.
- The CRC check is not optional. Noise regularly produces bit patterns that look like a frame, and every one of them would be a ghost station in the list. Measured here: ten seconds of pure noise produced four such candidates and not one of them survived the check.
- A failed check is two very different things, so they are counted apart. A candidate whose address field still holds twelve plausible callsign characters was a real transmission, damaged on the way — a collision, a fade — and is counted as DAMAGED; with DEBUG on, the line names who was lost. Anything else is noise between two flags and is counted as such. Random bytes pass the address test with a probability around $8 \cdot 10^{-11}$, and sixty seconds of pure noise produced not one. Measured on the air: in 152 s, 17 frames received, 5 damaged, 108 noise candidates — so the raw CRC count said far more about the noise floor than about lost stations.
- TRANSMIT sends a position beacon — BEACON keys PTT, the text field is its optional comment. The position is the centre of the Maidenhead locator from Settings > General, so use six characters or more: four is a 1°×2° field, roughly 111×70 km. It is sent as an uncompressed position without timestamp, path WIDE1-1, destination APZV71 — the APZ prefix marks experimental software, and claiming a registered one would misreport which program is on the air. 300 ms of flags precede the frame: too short and the receiving TNC loses the first bytes, which looks like a decoder fault at the far end.
- There is no digipeater, no APRS-IS gateway and no automatic repeat. A beacon that transmits by itself every few minutes is an operating decision, not a default.

Verified by loop-back — every beacon was decoded again by this program's own receiver, position, symbol, path and comment intact, and still error-free at 6 dB signal-to-noise. MIC-E was checked against known coordinates in both hemispheres, including a three-digit longitude with its offset. Decoding costs 0.2 ms per 20 ms of audio.

## Contact columns beside the console

On a wide screen the console leaves a few hundred pixels unused on either side. They hold a copy of the contact cards — the newest in the left column, the next ones in the right — so the stations heard stay in view while the panel itself is scrolled away. The tray background and the empty places are the panel's own, and the free places are numbered straight through both columns, large and faint, so an empty one says how far the list reaches.

- The heads carry the panel's name and, on the left, the total number of cards — not what fits, so the figure says whether anything is out of sight. The feet carry the two readings worth having in view: the sum of all talk timers on the left, the VOICE MATCH lamp with the number of learned voices on the right.
- The copies are fully operable: delete, log to Wavelog, start and stop the talk timer, mark a card, correct a callsign, and the hover detail. Every click is routed to the original card and performed by ITS button — cloning does not copy event handlers, and giving the copies their own would mean every behaviour existed twice, with the certainty of drifting apart one day.
- A copy shows the state of its original, not just its text: the current mark, the hold ring of a card picked by hand, the voice mark and the running talk time. Every move of the mark goes through one place that syncs the columns with it — a mark set by hand used to stop at the panel, because the only thing that refreshed the columns was the running clock, and clicking a card while no carrier is up leaves the clock stopped.
- One control in the title bar folds both columns away together; they are one display split over two columns, and leaving one open would look like a fault rather than a choice. The columns appear from 1620 px of window width (1220 for the console plus room for both) and never in the mobile deck, which has no margins to put them in.

## Callsign recognition (Vosk)

An optional, offline speech-recognition pass on the RX audio that detects spoken German callsigns; enable it in Settings > Audio. A grammar-constrained Vosk model (ITU/NATO phonetic alphabet plus German digits — the German spelling alphabet and letter names were dropped as their short, homophone-prone words caused most false matches) stays usable on noisy FM voice; the recognised letters are assembled into a callsign, restricted to the real German BNetzA allocation blocks (always 5–6 characters) and verified against the BNetzA Rufzeichenliste. Accuracy is raised by N-best rescoring — Vosk returns several hypotheses per over and the best callsign across them is chosen, preferring an assigned one — and by repetition voting: a listed call is shown at once, while an unassigned (VOID) hit must be heard twice within a short window before it appears, suppressing one-off mishears (operators send their call 2–3× anyway). A hit appears in a framed field in the title bar (coloured to the RX band) and as a toast, enriched from the offline list with the holder's name, address and licence class (A/E/N); a call that is not assigned is still shown but flagged VOID. Your own callsign is ignored, it runs only while the squelch is open, and it can also grade the mic-test audio. The callsign list is built once from the register with a converter (python -m app.callsign_list), which splits each entry into callsign, class, name, street, postcode and town, and writes the same list as JSON beside the cache for other tools (--json-only re-exports it in a second, without re-reading the PDF; the TSV stays what the service itself loads, being about 40% faster to read). The register is not consistent about its

separator — most entries use a semicolon between holder and address, a few hundred (club and relay stations) only a comma — so a comma in front of the postcode counts as a separator too, and a word hyphenated across a column break is joined again. QRZ.com is used only for a manual lookup in the logbook, never by the ASR.

### Region: Germany or the United States

Everything the recognition needs is country-specific, so it is switched in one place: Settings > General > Callsign region. The setting picks the speech model, the words operators spell with, the shape a callsign may have and the register it is checked against. Switching takes a moment — the model is reloaded and the register re-read — and the voice profiles are untouched, the speaker model knowing nothing of language.

- Germany: vosk-model-small-de-0.15, German digits, the BNetzA blocks (DA–DR plus two or three letters), verified against the Rufzeichenliste, classes A / E / N.
- United States: vosk-model-small-en-us-0.15, English digits, a callsign of K, N, W or A with a second letter A–L, then the region digit and one to three letters, verified against the FCC ULS register, classes Novice / Technician / General / Advanced / Extra. A single-letter prefix needs two suffix letters here (K1AB, not K1A): 1x1 calls are special-event and temporary, and allowing four-character reads would hand noise a whole class of false matches.
- Each language has one digit that exists only to survive a noisy channel, and both are in the grammar: "zwo" for 2 in German, "niner" for 9 in English. Leaving them out is not harmless — the word is then absorbed as unknown and the digit drops out of the callsign entirely.

Measured on the bench with synthesised speech, the American chain reads "kilo one juliet tango" as K1JT and pulls Joseph Taylor, Princeton NJ, Extra out of the FCC register; an invented call is flagged VOID as it should be. Two repetitions run together without a pause still merge into one over-long run, exactly as in German — on the air the gap between them separates them.

Powering the radio down suspends the recognition — there is no RX audio to analyse — and switching it back on resumes it. Your setting is not overwritten by this, so detection does not quietly stay off after a power cycle. The ASR indicator and the detected-callsign field stay on screen throughout; only the LED changes: green and pulsing while listening, red while suspended, dim when switched off. The pulse is what says “listening right now”, so neither off-state pulses.

## Band scan

Sweep a VHF/UHF range or the memory bank and see an occupancy spectrum + waterfall. Double-click a channel to tune the control VFO to it. 2 m and 70 cm are swept on the VFO; the memory bank scan recalls channels 0–199 and skips the empty ones.

The upper graph is the sweep running now, the lower one a history: ONE waterfall row per COMPLETED sweep, newest under the separator. A row therefore appears only when a full pass is done, and a pass takes as long as its channel count — 0.17 s per step, the time the audio needs to settle after a retune, or the level read would be the previous channel's. That is about 30 s for 2 m (160 steps), a minute for 70 cm (400) and a good two minutes for the air band (760). Both graphs read the AF level against a scale from −45 dBFS (nothing) to

−10 dBFS (loud), so a band with no activity paints a dim, even floor — a scanned but quiet band and one that has not been scanned yet must not look alike.

The air band cannot be swept that way: the VFO refuses to be tuned into 118–137 MHz over CAT, so the scan has to recall memory channels. It therefore writes its grid — 760 channels at 25 kHz — into memories 200–959 once and only recalls them afterwards. Before each run five of them are read back; if the grid is already there, nothing is written at all. The first version instead rewrote one scratch channel for every single step, which cost 760 flash writes on every sweep.

So: MEMORIES 200–959 BELONG TO THE AIR SCAN. Before the grid is written the panel asks, naming the range and what it will replace — there is nowhere to put 760 channels in the meantime, so this is the operator's decision, taken once: afterwards the grid is there and the question does not come back. It starts at 200 to leave the whole low bank alone: the quick keys are 0–16, or 50–66 while band A is in the air band, and 0–199 is what the memory-bank scan sweeps. Above it, 960–996 is free and 997–999 are the reserved scratch entries (997 for air-band tuning, 998 and 999 for the 2 m and 70 cm scans).

## ASR contacts

A panel below the band scan that collects every recognised station as an index card, so the last overs are readable at a glance instead of as a scrolling log. Cards sit in a tray of empty slots, newest first. The last 200 entries are kept on the Pi and restored when the panel opens; CLEAR ALL empties the view. In the mobile deck the panel has no tab — swipe to it past the band scan.

Each card carries:

- The callsign, in the largest and boldest type on the card — it is the identity of the contact. A zero is drawn slashed (DJØSH) so it cannot be misread as the letter O; that is display only, everything sent to the logbook keeps the plain 0.
- An avatar whose letters and colour are derived from the callsign itself (the characters after the region digit, plus a hue hashed from the whole call), so a station looks the same in every session without anything being stored.
- All three German licence classes A / E / N, with the holder's own one lit and the other two dimmed. If none is lit, the call is not in the BNetzA list.
- Name and town of the holder from the offline BNetzA list — never from QRZ.com, which the ASR does not query. The town carries its postcode in front of it, as on an envelope; the street stays behind the card and travels to the logbook.
- The time the station was last heard and its talk timer, pinned to the bottom edge of the card.

A station heard again does not get a second card. The existing one flashes, is marked, and counts up (×2, ×3 …) — but it stays where it is, so the tray does not reshuffle under you on every over. Exactly one card carries the red border at a time: the most recently heard. The ordering therefore follows first contact, not last mention.

Hovering a card shows the recogniser detail that used to fill the log lines: word confidence (0.00–1.00, the mean over the individual spelled letters — a statement about the acoustics, not about whether the callsign is real), the S-value and receiving band at the moment of detection, the raw words Vosk actually heard, the rejected N-best candidates, and any

5/6-character correction that was applied.

Two buttons per card: the play symbol logs the QSO straight to Wavelog with the name from the BNetzA list pre-filled, and turns teal once it has gone through so nothing is sent twice. The cross removes a misrecognised contact. That deletion happens on the Pi, not just in the browser: the card is dropped from the log buffer (otherwise it would return the next time the panel opens), every connected client loses it at once, and the callsign is released from the 90-second de-dupe window so a corrected reading can be reported immediately.

Talk time. The clock face in the bottom row of a card is the start/stop button for that station's timer — and the count also runs by itself: it starts on the rising edge of BUSY and stops on the falling one, so an over is timed without touching anything. The clock always belongs to exactly one card, the marked one. Clicking a card moves the mark, and with it the timer, onto that contact — for when the recognition attributed an over to the wrong station. Every card keeps its own total: switching the focus banks the running stretch on the card it came from and picks up the new card's figure, so nothing is lost or counted twice. The display is fixed at hh:mm:ss, otherwise the button would change width as it counts.

The panel head carries the sum of all timers (Σ) and five controls:

- The input field takes a callsign by hand when the recognition misses one — upper-case letters and digits only, filtered while typing. LOG (or Enter) creates the card through the backend exactly like a recognised one, so it lands in the history and reaches every connected client. A message says whether the BNetzA list knew the call.
- MOD marks the focused card as net control. Its timer keeps running and stays readable on the card, but it is left out of every figure: neither the Σ total nor the ranking counts it, and it does not set the scale of the bars — without that, the station holding the round together would flatten every other bar. A flagged card wears a ring around its avatar. The flag lives in the browser (local storage), not on the Pi: it belongs to whoever is listening to this round, and it survives a reload.
- STATS opens the evaluation: every card by talk time, longest first, with callsign, name, a bar and the time. The bar is scaled against the longest over rather than against the sum. Moderators are listed below the ranking, without a bar and without a rank. COPY puts the list on the clipboard as plain text. The dialog keeps counting while a station is being heard, so it can stay open through an over.
- CLEAR ALL empties the log — in dark red, because it takes every card at once, on the Pi and in every open client, and asks before it does. The talk times go with them (they live in the browser); the learned voices do not — clearing a list is not the same statement as “this was wrong”, which is what the cross on a single card means.
- VOICE MATCH, at the right-hand end, switches the speaker recognition on and off (see below). Its lamp pulses green while it is listening; an outlined button means it only observes, a filled one that it acts.

Correcting a callsign: click the call on the card and type over it (Enter applies, Escape and clicking away cancel). The card keeps its talk time, its repeat count and its place, name and town are re-read for the corrected call, and the change reaches every open client. If the corrected call already has a card, the two are merged and their times added. The voice profile is renamed with it — otherwise a mis-heard call would keep collecting a voiceprint under a name that never existed while the real station never gets one.

The interface is in English throughout — labels, hints, messages and error texts. This

manual exists in German as well; the labels quoted in it are the English ones, because those are what is on screen.

## Voice ID — an over without a callsign

Not every over carries a spoken callsign, and the talk timer then counts onto whoever was marked last. Speaker recognition closes that gap: the voiceprint of an over is compared against the stations whose callsign HAS been heard. The profiles build themselves while the panel is simply used — but not from every over that carries a call: which call belongs to the SPEAKER is the whole question (see the enrolment rule below).

- An over ends at the PAUSE in speech, not at the carrier. Through a repeater BUSY stays up for the whole QSO, so the falling edge never comes and every station would land in one segment with their voices averaged together. The pause boundary works in simplex too, where the carrier drop simply arrives first.
- Under about 3 s of speech nothing is guessed at: the over is skipped.
- Two stages. Observe only decides and reports — the verdict appears above the card tray and in the card's hover detail, and nothing moves. Assign additionally moves the mark and the talk timer onto the recognised station; such a card is drawn with a dashed border so it is never confused with the solid red of “just heard saying its call”. The stage is chosen either in the panel — ASSIGN sits beside VOICE MATCH in the head, filled while it is intervening, outlined while it only reports — or in Settings > Audio > Voice ID. Switching ASSIGN on switches the recognition on with it: a stage that can act on nothing would be a dead switch. Switching it off leaves the recognition running and falls back to observing.
- A card picked BY HAND always wins: while a manual selection holds, the voice moves neither the mark nor the timer. It is released when the next over begins, so the correction applies to the over it was made in. A callsign actually heard still moves the mark — that is evidence, not a guess.
- It needs the callsign recognition switched on (it listens to the same audio tap), and it costs about 1.3 s of one core per over, afterwards.

### Which callsign may become a profile

The first version of this learned from any over that carried exactly one callsign, and the profiles of different stations came out 0.66–0.84 alike — too close to tell apart. The fault is not in the maths but in operating practice: the callsign spoken in an over is very often the OTHER station's. “DL1ABC, are you there?” files the caller's voice under the called station, and two such overs are enough to smear a profile into a mixture of two people.

Where the callsign falls in the over is the signal that separates the two cases. An operator signs off with their own call and names the other one when calling up. A label is therefore accepted only:

- in the last 40 % of the over, or within its final 3 seconds — the sign-off position;
- from at least 4 s of speech: a profile is the yardstick every later over is measured against, so it is held to more than a match is;
- for a callsign the register actually knows, so a mishear cannot become a phantom profile;
- never for your own callsign.

Two callsigns in one over — the everyday “DL1ABC from DL2XYZ” — are read in ITU order, where the called station comes first and the speaker signs last. That is a convention, not evidence, so it is accepted only in its clean form (exactly two calls, the other one confined to the first half) and counted separately as “by order” in the panel's statistics: how much of the learning rests on a convention stays visible, and the whole line names every reason an over was skipped rather than only counting them.

A card picked BY HAND is exempt from all of it — it is a person saying whose voice this is, where everything above only infers it. Neither length, nor position, nor the register is tested against a pick, and of two picks in one over the later one wins. A pick during an over labels that over; up to 12 s after one it labels the over that just ended, because the correction usually arrives a moment too late.

Measured on two off-air recordings of the same round (14 + 11 overs, four stations in both): the same station across recordings scored 0.62–0.79 cosine, different stations 0.33 on average and 0.64 at worst. At a threshold of 0.55 with a 0.05 margin over the runner-up, 3 of the 4 known stations were identified, none wrongly, and none of the 7 stations unknown to the profile set was mistaken for a known one. The margin does the real work: a stranger's best match is flat (0.01–0.08 ahead of the second), a true match stands clear (0.20–0.34). Treat it as an assistant that may abstain, not as proof — and never log a QSO on a voice match alone.

The model is Vosk's own speaker model (vosk-model-spk-0.4, ~14 MB), an x-vector trained on 8 kHz telephone speech — narrowband, noisy, channel-varied, which is far closer to FM off a repeater than a wideband model would be. A voiceprint is biometric data: the profiles stay on the Pi in speakers.json (gitignored, never uploaded) and a profile is dropped with its card.

## Logbook (Wavelog + QRZ.com)

Logs QSOs to a locally installed Wavelog instance. Enter just the callsign (and optionally a name) — frequency, band, mode, date/time and your own callsign are filled in automatically from the live control band and station profile. LOOKUP fetches the operator's name, grid, QTH, country and e-mail from QRZ.com (XML data API) and 'worked before' / DXCC from Wavelog. The ADDRESS field beside QTH takes the holder's street and postcode; logging a contact straight from an ASR card fills it in from the register, and it travels as the ADIF field ADDRESS while QTH keeps the plain town, so Wavelog's statistics stay clean. STATE beside it carries the US state as the ADIF field of the same name — Wavelog counts states for WAS, and a state buried in the address line would not be counted. It stays empty in Germany. LOG QSO sends the contact as ADIF. A green dot shows when Wavelog is reachable; the panel lists the most recent QSOs (with the looked-up details, deletable individually or via CLEAR) and Wavelog's QSO counts (today/month/year/total). Configure the Wavelog URL, API token and station profile, and the QRZ.com username/password, in Settings > Logging. Credentials are stored only on the Pi (runtime.json), never committed.

## Settings

Tabs: General (callsign, locator, callsign region, API backend URL, serial port/baud, GPIO power, auto power-off, logo, GitHub self-update, Root-CA download), Audio (device, USB

mixer, voice filters, test tone, audio timing), Rig-Info, Rig-Memory, Rig-DTMF, Logging (Wavelog + QRZ.com), Panels (which panels this browser shows) and Pi-Hardware (host metrics, and the backend's own resident size beside them).

### Panels

A shack without a HackRF, or one that never runs digimodes, carries two screens of furniture around. Settings > Panels lays the panels out as a small map of the console — one tile each, two to a row where the console puts two (the VFOs, then PTT beside audio), in the same order top to bottom, and a tile with its switch off is dimmed, so the map shows the console you are going to get. Switching one off takes it out of the desktop console, and in the app its tab leaves the bottom bar with it, so the swipe order closes up behind it instead of snapping onto an empty slide. Nothing is unloaded: a hidden panel keeps taking its status updates and is live again the moment it is switched back on, so this is a view setting rather than a switch-off. The one exception is the HackRF, where hiding the panel also stops the sweep, because that one costs the Pi real work. The choice is stored in the browser, like the collapse arrows beside it: the phone in the car and the PC in the shack may show different consoles, and SHOW ALL brings everything back.

### Memory

The backend starts at roughly 320 MB — mostly the callsign list and, with recognition on, the two Vosk models. With nobody listening it stays there: measured over ten hours with the sound card running the whole time, 361 MB start to finish, give or take a megabyte. Settings > Hardware shows its resident size next to the host's, and the 24 h graph below answers what a single reading cannot: whether it has been climbing all night.

A connected listener used to cost about 45 MB an hour, which is what filled up the 1 GB Pi behind the original report. It is fixed, and the chase is worth recording because nearly every intuition about it was wrong. Switching recognition off changed nothing. A bare WebRTC loopback in an empty process grew just as fast while gaining two Python objects in five minutes, so nothing reachable was holding it. Three allocators grew identically, so it was not fragmentation. Rebuilding the codec context every minute reclaimed none of it. Mono instead of stereo changed nothing on its own. But the same loop with pcm_s16le in place of Opus was flat to the byte, and libopus called directly grew by 0.03 MB a minute instead of 0.23.

So the leak is in the library wrapper around Opus that comes with PyAV, not in Opus and not in this code. On top of that, aiortc's encoder hard-codes a stereo layout and resamples the radio's mono there, although the offer already pins Opus to one channel: half the growth was that resample, for a second channel nobody asked for. The backend therefore encodes mono straight against libopus, bypassing both. Measured end to end through a real peer connection with a receiver decoding every frame: 0.42 down to 0.02 MB a minute.

A third of it was left, and only watching the live curve overnight found it: the same resampler on the way back. The mic path built one per listener, to turn the decoder's stereo output into the radio's mono, and ran it fifty times a second whether or not anyone was transmitting — 13 MB an hour for every connected browser, sitting there while the station was idle. What it was doing is a channel downmix at an unchanged rate and format, which is three lines of numpy. The slope scaling with the number of listeners is what gave it away:

one listener 19.2 MB/h, two 26.5 — and after the fix, with two listeners, minus 3.9 MB/h over 22 minutes. Negative, because the five-minute reclaim now wins. The leak had three sites, and together they are what filled a 1 GB Pi overnight.

Two things keep the rest bounded. Housekeeping hands freed memory back every five minutes (about 7 ms a run). And the allocator is capped to two arenas at startup: the C library otherwise hands each thread its own 64 MB arena, and this process had grown thirty of them, which cut the resident size from 611 MB to 318 MB once capped. The systemd unit still adds a memory ceiling with an automatic restart, so a runaway costs a restart instead of a frozen board.

# 8 Mobile App (PWA)

The UI installs as a Progressive Web App: full-screen, with an app-shell service worker for instant launch. On phones the panels become a vertical swipe deck — swipe up/down, one panel per screen (this keeps the deck's scroll axis off the horizontal sliders, so they stay usable) — with the title bar as a slim vertical strip on the left and an icon tab rail on the right. Past the last panel is an info page listing the app version and browser/environment details. The app is forced to landscape; portrait shows a rotate hint. Install via the browser menu (Install / Add to Home Screen); on iOS use Safari > Share.

While the app is open the phone screen is kept awake via the Screen Wake Lock API, so it won't dim or lock mid-QSO. The browser audio link reconnects automatically after a brief network interruption, and if the connection to the backend is lost while transmit is latched, the PTT is released (locally and by a backend watchdog) so the rig can never stay keyed unattended.



# 9 Trusted Certificate (Root CA)

A self-signed certificate is fine on the desktop, but mobile browsers will not install the PWA or run the service worker without a trusted certificate. Create your own root CA, sign the server certificate with it, and trust the CA on the phone. A download link for the CA appears in Settings > General once it exists.

```
cd backend/certs && mkdir -p ca
# 1) Root CA (10 years) — keep ca.key secret
openssl genrsa -out ca/ca.key 4096
openssl req -x509 -new -key ca/ca.key -sha256 -days 3650 \
  -subj "/CN=TM-V71 Remote Root CA" \
  -addext "basicConstraints=critical,CA:TRUE,pathlen:0" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" -out ca/ca.crt
# 2) server cert signed by the CA (list every name/IP in the SAN)
openssl genrsa -out key.pem 2048
openssl req -new -key key.pem -subj "/CN=tm-v71.example.lan" -out s.csr
openssl x509 -req -in s.csr -CA ca/ca.crt -CAkey ca/ca.key \
  -CAcreateserial -days 825 -sha256 -extfile leaf.ext -out cert.pem
sudo systemctl restart tmv71-remote.service
```

Install ca/ca.crt on the phone (Settings > Security > Install a certificate > CA certificate). The leaf is valid 825 days; re-issue it from the same CA without re-installing on phones. Keep ca/ca.key secret.

# 10 Configuration

Settings are read from environment variables (prefix TMV71_) or a .env file, with web-UI changes persisted to backend/app/runtime.json. Key variables:

```
TMV71_SERIAL_PORT=/dev/ttyUSB0   TMV71_SERIAL_BAUD=57600
TMV71_HOST=0.0.0.0               TMV71_PORT=8443
TMV71_AUDIO_DEVICE=NAD           TMV71_AUDIO_ENABLED=true
TMV71_GPIO_POWER_PIN=17          TMV71_CALLSIGN=DJ0SH
TMV71_ASR_MODEL_DIR=...          TMV71_ASR_CALLLIST_PDF=...
TMV71_SSL_CERTFILE=...  TMV71_SSL_KEYFILE=...
```

# 11 REST & WebSocket API

## Control & status

```
GET  /api/status            live radio state
POST /api/frequency         set VFO frequency
POST /api/band-mode         VFO / memory / call
POST /api/control-band      select control band
POST /api/ptt               key / un-key (CAT)
POST /api/ptt-band          select TX band
POST /api/squelch /step     squelch level / step
POST /api/vfo               shift/offset/tone/bw
GET  /api/info /version     rig + app info
WS   /ws                    live status stream
```

## Memory channels

```
GET    /api/memories?start&end   list channels
GET    /api/memories/{ch}        one channel
PUT    /api/memories/{ch}        write channel
DELETE /api/memories/{ch}        clear channel
GET    /api/memories.csv         export CSV
POST   /api/memories/import      import CSV
POST   /api/recall               recall to band
```

## Audio

```
POST /api/webrtc/offer      WebRTC SDP offer
GET  /api/audio/status      RX/TX levels, flags
GET  /api/audio/devices     list sound devices
POST /api/audio/device      pick device
POST /api/audio/gain        rx/tx gain + TX AGC
POST /api/audio/buffer      tx buffer / ptt tail / rx buffer
POST /api/audio/tones       roger/test/mic/lowpass/de-emph/squelch
POST /api/audio/record[/clear]  raw RX recorder
GET  /api/audio/record.wav  download recording (WAV)
GET/POST /api/audio/mixer   USB card mixer
```

## Digimodes & selcall

```
GET  /api/digi              CW/RTTY/POCSAG/APRS status
POST /api/digi/config       mode + parameters
POST /api/digi/tx           encode + transmit
POST /api/digi/decode-recording  decode the RX buffer
WS   /ws/digi               decoded text stream
GET/POST /api/asr/config    callsign recognition (Vosk)
POST /api/asr/log           add a callsign by hand
PATCH /api/asr/log/{call}   correct a callsign (profile follows)
GET/POST /api/asr/speaker   voice ID: on/off, stage, limits
DELETE /api/asr/log         empty the contact log (CLEAR ALL)
DELETE /api/asr/log/{call}  drop a misrecognised contact
```

```
WS   /ws/callsign           recognised-callsign events
GET  /api/selcall           5-tone status
POST /api/selcall/config    standard / tone / own
POST /api/selcall/tx        send a 5-tone call
WS   /ws/selcall            decoded calls stream
```

## SDR & scan

```
GET  /api/hackrf            SDR status
POST /api/hackrf/start|stop|config
WS   /ws/hackrf             spectrum/waterfall frames
GET  /api/scan  POST /api/scan/start|stop  band scan
```

## Logbook

```
GET/POST /api/log/config     logbook credentials (Wavelog + QRZ)
POST /api/log/test           test the Wavelog connection
POST /api/log/qrz/test       test the QRZ.com login
GET  /api/log/stations       Wavelog station profiles
POST /api/log/lookup         callsign lookup (QRZ + Wavelog)
POST /api/log/qso            log a QSO
GET  /api/log/recent         recent QSOs + online + Wavelog stats
POST /api/log/recent/delete  delete one recent entry
POST /api/log/recent/clear   clear the recent list
```

## System & power

```
GET/POST /api/power-switch   GPIO power
POST /api/gpio-config        set GPIO pin
POST /api/auto-power-off     idle auto-off
GET/POST /api/serial-config  serial port/baud
GET/POST /api/callsign /theme
GET  /api/system             Pi host metrics
GET/POST /api/update         GitHub self-update
```

# 12 Troubleshooting

- No CAT / 'radio offline': check the serial port and baud in Settings; the FTDI cable must be on /dev/ttyUSB0 (or set the right port).
- No audio: ensure HTTPS, click CONNECT and allow the mic; check the USB card and its mixer levels (playback drives the radio mic on TX).
- PWA won't install / theme not switching on a phone: the certificate is not trusted — install the Root CA (chapter 9).
- RX filter only on one channel: reconnect audio (Disconnect/Connect) to renegotiate Opus to mono.
- Decoders need a clean signal; tune levels and (for RTTY) the mark tone.

# 13 Security

LAN-only by design; there is no authentication. Do not expose the port directly to the internet — use a VPN (WireGuard/Tailscale) or a reverse proxy with TLS + auth. The Root CA private key never leaves the Pi.

# 14 Credits & License

Kenwood PC protocol docs: LA3QMA/TM-V71_TM-D710-Kenwood. Built with aiortc (WebRTC/Opus) and sounddevice. Fonts: Saira, IBM Plex Mono, DSEG (7-segment), Neuropol (title). See the repository for license details.